

个人作业报告

题目：**并行程序设计与算法**

理论课第四次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	吴迪

2026 年 4 月 30 日

【摘　要】

此作业报告是并行程序设计与算法课程的**第四次作业**，作业题目来自智算习堂[1]的作业内容。

关键词：　并行程序设计与算法

目录

1 第一题 1
1.1 题目 1
1.2 答案 1

2 第二题 3
2.1 题目 3
2.2 答案 3

3 第三题 5
3.1 题目 5
3.2 答案 5

4 第四题 7
4.1 题目 7
4.2 答案 7

5 第五题 9
5.1 题目 9
5.2 答案 9

参考文献 11

1 第一题

1.1 题目

并行化时我们需要注意并识别循环依赖，分析以下两个 C 语言循环片段，判断其是否可以直接使用 `#pragma omp parallel for` 进行并行化。如果不能，请说明原因：

片段 A：

```
for(i= 1; i< n; i++){
    A[i]= A[i-1]+ B[i];
}
```

片段 B：

```
int index= 0;
for(i= 0; i< n; i++){
    index=(i+ offset)% n; //offset为一个固定的常数
    C[index]= A[i]* B[i];
}
```

1.2 答案

1.2.1 片段 A 的并行化分析

片段 A **不能**直接使用 `#pragma omp parallel for` 进行并行化。

原因：存在**循环携带依赖**（Loop-carried dependence）。第 i 次迭代计算 $A[i]$ 时需要依赖第 $i-1$ 次迭代计算出的 $A[i-1]$ 值。这种数据依赖关系使得迭代之间无法独立执行，必须按顺序串行执行。

1.2.2 片段 B 的并行化分析

片段 B **在理论上可以并行化**，但在实际使用 `#pragma omp parallel for` 时可能存在问题。

原因：表达式 $index = (i + offset)\%n$ 实际上是对数组索引的一种置换（permutation），在数学上保证了不同的 i 会映射到不同的 $index$，因此不会发生写冲

突。

然而，编译器可能无法在编译时证明这种映射关系是无冲突的，因为这需要进行复杂的数学推理分析来保证不会发生冲突，因此从实际使用的角度出发，不一定能够安全地自动并行化。

2 第二题

2.1 题目

请阅读以下使用 OpenMP 并行化的 C 语言代码片段，假设 n 非常大，线程数为 4 个：

```
double result= 0.0;
//使用 OpenMP并行化
#pragma omp parallel for reduction(+: result)
for(int i= 0; i< n; i++){
    result+= HeavyCompute(i);
}
```

其中 HeavyCompute(i) 是一个模拟函数，i 越大，计算所需的时间越长。

回答下列问题：

1) 虽然这段代码计算结果是正确的，但在性能上可能会出现什么问题？
2) OpenMP 的 parallel for 默认使用哪种调度策略？为什么这种策略在上述场景下效率低下？
3) 请修改 #pragma 指令，使用一种更合适的调度策略来平衡负载，提高程序运行速度。

2.2 答案

2.2.1 性能问题分析

虽然代码计算结果正确，但在性能上会出现**负载不均衡**（Load Imbalance）问题。

由于 HeavyCompute(i) 的计算时间随 i 增大而增加，而默认的静态调度策略会将连续的迭代块分配给各个线程，导致处理较大 i 值的线程需要更长时间完成，而处理较小 i 值的线程会提前完成并处于空闲状态，造成 CPU 资源浪费。

2.2.2 默认调度策略及效率问题

OpenMP 的 `parallel for` 默认使用**静态调度**（static scheduling）策略，通常将迭代空间均匀地划分为连续的块（chunk size 约为 $\frac{n}{\text{线程数}}$）分配给各个线程。在这种场景下效率低下的原因：

- 静态调度无法适应不同迭代计算量的差异
- 计算量大的迭代集中在某些线程，造成这些线程成为性能瓶颈
- 其他线程完成工作后只能等待，无法帮助处理剩余的重负载
- 整体执行时间由最慢的线程决定，无法充分利用所有 CPU 核心

2.2.3 改进的调度策略

方案一：使用动态调度

修改 `#pragma` 指令：

```
#pragma omp parallel for reduction(+: result) schedule(dynamic, chunk_size)
```

其中 `chunk_size` 可以根据实际情况调整，通常设置为较小的值（如 1、10 或 100）以获得更好的负载均衡。动态调度会在运行时将迭代块动态分配给空闲线程，每个线程完成当前块后会请求新的任务块。**较小的 chunk_size 能实现更精细的负载均衡**，但会增加调度开销；较大的 chunk_size 减少调度开销但可能降低均衡效果。建议根据 HeavyCompute(i) 的计算时间差异程度选择合适的 chunk_size。

方案二：使用指导性调度

```
#pragma omp parallel for reduction(+: result) schedule(guided)
```

指导性调度是一种自适应的动态调度策略，初始时分配较大的任务块以减少调度开销，随着迭代进行逐渐减小块大小以提高负载均衡。它自动平衡调度开销和负载均衡，特别适合计算时间差异大且不规律的场景，无需手动调整 chunk_size 参数。

这两种策略都能让线程在完成当前任务后动态获取新的任务，从而更好地平衡负载。

3 第三题

3.1 题目

请阅读以下代码片段并回答问题：

```
#include<stdio.h>
#include<omp.h>
int main(){
    int i, s= 0, a= 5;
    #pragma omp parallel num_threads(3) private(i) shared(a) reduction(+:s) {
        int tid= omp_get_thread_num();
        a+= tid;
        #pragma omp for
        for(i= 1; i<= 6; i++){
            s+= i;
        }
    }
    printf("s=%d\n", s);
    printf("a=%d\n", a);
    return 0;
}
```

回答下列问题：

1) 变量 i, s, a 分别是什么变量?
2) 指出这段代码所存在的问题。
3) 分析最终 a 和 s 的输出。

3.2 答案

3.2.1 变量类型分析

- **i**: 私有变量（private）。在并行区域内每个线程都有自己的副本
- **s**: 归约变量（reduction variable）。每个线程有私有副本，最后通过加法归约合并
- **a**: 共享变量（shared）。所有线程共享同一个内存位置

3.2.2 代码存在的问题

代码存在**数据竞争**（Data Race）问题：

变量 a 被声明为共享变量，但多个线程同时执行 `a += tid` 操作。由于这个操作不是原子的，会导致数据竞争，产生不确定的结果。

具体来说，三个线程（$\text{tid} = 0, 1, 2$）都会同时读取、修改和写入共享变量 a，可能导致某些增量操作丢失。

3.2.3 最终输出分析

s 的输出：$s = 21$

原因：归约操作保证了正确性。循环 `for(i = 1; i <= 6; i++)` 的总和为 $1 + 2 + 3 + 4 + 5 + 6 = 21$，归约操作会正确地将所有线程的局部和相加。

a 的输出：**不确定**

理论上，如果没有数据竞争，a 应该等于 $5 + 0 + 1 + 2 = 8$。但由于存在数据竞争，实际结果通常为 6、7 或 8，取决于线程执行的交错顺序。

4 第四题

4.1 题目

某程序使用 OpenMP 并行化如下循环：

```
#pragma omp parallel for num_threads(4)
for(int i= 0; i< 16; i++){
    work(i);
}
```

设系统采用默认调度方式，并假设每次迭代 work(i) 的执行时间基本相同。

回答下列问题：

1) 若 16 次迭代被连续地、尽可能均匀地分配给 4 个线程，请写出每个线程可能负责的迭代编号范围。
2) 这种分配方式有什么优点？
3) 如果不同迭代的计算量差异非常大，那么默认这种分配方式可能会带来什么问题？

4.2 答案

4.2.1 迭代分配范围

在默认静态调度下，16 次迭代被均匀分配给 4 个线程：

- 线程 0：迭代 0, 1, 2, 3（范围：0-3）
- 线程 1：迭代 4, 5, 6, 7（范围：4-7）
- 线程 2：迭代 8, 9, 10, 11（范围：8-11）
- 线程 3：迭代 12, 13, 14, 15（范围：12-15）

4.2.2 分配方式的优点

这种静态连续分配的优点包括：

- **良好的空间局部性**：连续的内存访问模式有利于缓存性能
- **低调度开销**：在进入并行区域时就确定了分配方案，无需运行时调度
- **可预测性**：每次运行的分配结果一致，便于调试和性能分析

- **适合均匀负载**：当各迭代计算量相近时，能实现良好的负载均衡

4.2.3 计算量差异大时的问题

当不同迭代的计算量差异很大时，静态分配会导致**严重的负载不均衡**：

- 某些线程可能分配到计算量很大的迭代，需要很长时间完成
- 其他线程完成自己的轻量级迭代后只能空闲等待
- 整体执行时间由最慢的线程决定，无法充分利用所有 CPU 核心
- 并行效率显著降低，甚至可能比串行执行还要慢（考虑并行开销）

5 第五题

5.1 题目

阅读下面的 OpenMP 程序片段：

```
double x= 0.0;
double y= 0.0;
#pragma omp parallel num_threads(4) {
    for(int i= 0; i< 1000; i++){
        #pragma omp atomic
        x+= 1.0;
        #pragma omp atomic
        y+= 1.0;
    }
}
```

回答下列问题：

1) 这两条 atomic 指令分别保护的是什么操作?
2) 对变量 x 的更新和对变量 y 的更新，是否会被视为同一个” 临界区”? 为什么?
3) 如果把两条 atomic 改成一个 critical 代码块，将两次加法都放进去，程序的并发性会发生什么变化?

5.2 答案

5.2.1 atomic 指令保护的操作

两条 atomic 指令分别保护以下操作：

- 第一条：保护对变量 x 的读-修改-写操作（$x+ = 1.0$）
- 第二条：保护对变量 y 的读-修改-写操作（$y+ = 1.0$）

每个 atomic 指令确保对应的加法操作是原子执行的，防止多个线程同时修改同一变量导致的数据竞争。

5.2.2 临界区分析

对变量 x 的更新和对变量 y 的更新**不会**被视为同一个临界区。

原因：OpenMP 的 atomic 指令是针对特定内存位置的。每个 atomic 指令只保护对其指定变量的访问。由于 x 和 y 是不同的内存位置，它们的 atomic 操作是独立的，可以并发执行。一个线程在执行 $x+ = 1.0$ 的同时，另一个线程可以执行 $y+ = 1.0$，两者互不影响。

5.2.3 使用 critical 代码块的影响

如果将两条 atomic 指令改为一个 critical 代码块：

```
#pragma omp critical
{
    x += 1.0;
    y += 1.0;
}
```

程序的并发性会**显著降低**：

- 原来的 atomic 版本允许对 x 和 y 的更新并发进行
- critical 版本强制所有线程串行执行整个代码块
- 即使不同线程操作不同的变量，也必须排队等待
- 并发度从可以同时更新两个变量降低到只能串行执行
- 性能会明显下降，特别是在高并发场景下

因此，在保护不同变量的情况下，使用单独的 atomic 指令比使用 critical 代码块更高效。

参考文献

[1] [Z]. https://easyai.sysu.edu.cn.